From Python to Polynomial Certificates

A Concrete Barrier-Theoretic Verification
of Interprocedural Taint Safety

SQL Injection · Taint Lattices · Compositional Barriers

A3 Python Verification Framework

March 2026

Abstract

We present a complete, concrete example of mapping Python source code through bytecode to a polynomial transition system, and synthesizing a barrier certificate that proves freedom from SQL injection. This is a property where barrier certificates are *strictly more powerful* than type checking, standard abstract interpretation, or bounded model checking: the barrier provides a compositional, interprocedural separation proof that no finite unrolling or conservative type-based analysis can replicate. We give every equation, every state variable, and every diagram, from the Python source to the final SOS certificate.

Contents

1 The Running Example: SQL Injection with Sanitization **3**
1.1 Python Source Code . . . 3
1.2 The Verification Challenge . . . 3

2 Step 1: Python → Bytecode → Control Flow Graph **4**
2.1 Bytecode Extraction . . . 4
2.2 Control Flow Graph . . . 4

3 Step 2: Transition System Encoding **4**
3.1 State Space . . . 4
3.2 Initial States . . . 5
3.3 Transition Relation . . . 5
3.4 Unsafe Region . . . 6
3.5 Transition System Diagram . . . 6

4 Step 3: Barrier Certificate Synthesis **6**
4.1 The Barrier Function . . . 6
4.2 Verification of the Three Conditions . . . 7
4.3 Barrier Separation Diagram . . . 8

5 Step 4: SOS Certificate **8**
5.1 From Barrier to Sum-of-Squares . . . 8
5.2 Concrete SOS Decomposition . . . 9

6 The Unsafe Variant: Certificate Refutation **9**

7 Why Barriers Succeed Where Others Fail **10**
7.1 Comparison Table 10
7.2 The Fundamental Limitation of Non-Barrier Approaches 10
7.3 Compositional Barrier Diagram 11

8 Complete Formal Summary **11**
8.1 Contrast: Why This Cannot Be Done Without Barriers 12

9 Connection to the A3 Codebase **12**

1 The Running Example: SQL Injection with Sanitization

1.1 Python Source Code

Consider the following web-application function that queries a database using user-supplied input:

Listing 1: Target Python function

```python
def search_users(request, cursor):
    """Search users by name from HTTP request."""
    raw_name = request.args.get("name")       # TAINT SOURCE
    if raw_name is None:
        return []

    clean_name = escape_string(raw_name)       # SANITIZER

    query = "SELECT * FROM users WHERE name = '%s'" % clean_name
    cursor.execute(query)                      # SINK
    return cursor.fetchall()
```

The question: *Is this function safe from SQL injection (CWE-089)?*

There is also an unsafe variant:

Listing 2: Unsafe variant (no sanitizer)

```python
def search_users_unsafe(request, cursor):
    raw_name = request.args.get("name")        # TAINT SOURCE
    if raw_name is None:
        return []

    query = "SELECT * FROM users WHERE name = '%s'" % raw_name
    cursor.execute(query)                      # SINK: VULNERABLE!
    return cursor.fetchall()
```

1.2 The Verification Challenge

Why Non-Barrier Approaches Fail

Type checking cannot distinguish a tainted `str` from a clean `str`. Both have type `str`; the type system has no refinement to capture "sanitized."

Standard abstract interpretation with a taint domain must conservatively havoc at unknown function calls. When `escape_string` is an opaque library function, the abstract interpreter must assume the return value *might* still be tainted (it cannot prove the function removes taint without analyzing it).

Bounded model checking (BMC) can unroll a fixed number of paths, but:

1. It cannot prove safety for *all* inputs (only for explored paths).
2. Interprocedural BMC requires inlining all callees, which does not scale.
3. It provides no reusable proof artifact.

Barrier certificates provide a *compositional separation proof*: a polynomial function B over the state variables that is non-negative on all reachable states and negative on all unsafe states. The barrier composes across function boundaries via assume-guarantee contracts, needs no inlining, and serves as a checkable proof artifact.

2 Step 1: Python → Bytecode → Control Flow Graph

2.1 Bytecode Extraction

The A3 framework compiles the source to CPython 3.12+ bytecode and extracts a control flow graph. The relevant bytecode for Listing 1 (simplified) is:

Listing 3: Simplified bytecode for `search_users`

```
  0: LOAD_FAST         request            # push request
  2: LOAD_ATTR         args               # request.args
  4: LOAD_ATTR         get                # request.args.get
  6: LOAD_CONST        "name"
  8: CALL              1                  # raw_name = request.args.get("name")
 10: STORE_FAST        raw_name
 12: LOAD_FAST         raw_name
 14: POP_JUMP_IF_NOT_NONE  20             # if raw_name is None:
 16: BUILD_LIST        0                  #   return []
 18: RETURN_VALUE
 20: LOAD_GLOBAL       escape_string      # clean_name = escape_string(raw_name)
 22: LOAD_FAST         raw_name
 24: CALL              1
 26: STORE_FAST        clean_name
 28: LOAD_CONST        "SELECT...%s"
 30: LOAD_FAST         clean_name
 32: BINARY_OP         MOD                # query = "..." % clean_name
 34: STORE_FAST        query
 36: LOAD_FAST         cursor
 38: LOAD_ATTR         execute
 40: LOAD_FAST         query
 42: CALL              1                  # cursor.execute(query)
 44: POP_TOP
 46: LOAD_FAST         cursor
 48: LOAD_ATTR         fetchall
 50: CALL              0
 52: RETURN_VALUE
```

2.2 Control Flow Graph

3 Step 2: Transition System Encoding

3.1 State Space

Following the `step_relation.py` module of the A3 framework, we model the machine state as:

Definition 3.1 (Taint-Augmented Machine State)**.** A state $s \in \mathcal{S} \subseteq \mathbb{R}^5$ is a tuple:

$$s = (\pi,\ \tau,\ \kappa,\ \nu,\ \rho)$$

where:

- $\pi \in \{0, 1, \ldots, 5\}$ is the *program counter* (basic block index),
- $\tau \in [0, 1]$ is the *taint level* of the data flowing toward the SQL sink (0 = clean, 1 = tainted by untrusted source),
- $\kappa \in [0, 1]$ is the *sanitization flag* (1 = sanitizer has been applied, 0 = not yet),
- $\nu \in \{0, 1\}$ is the *null flag* (1 = `raw_name is None`, 0 = not None),
- $\rho \in \{0, 1\}$ is the *parameterization flag* (1 = query uses parameterized execution, 0 = string formatting).

Remark 3.2. The key lifting from discrete types to $\mathbb{R}^n$: Python's `str` type carries no taint information, but our *semantic* state augments each value with a real-valued taint coordinate τ. This is the information that type systems cannot express.

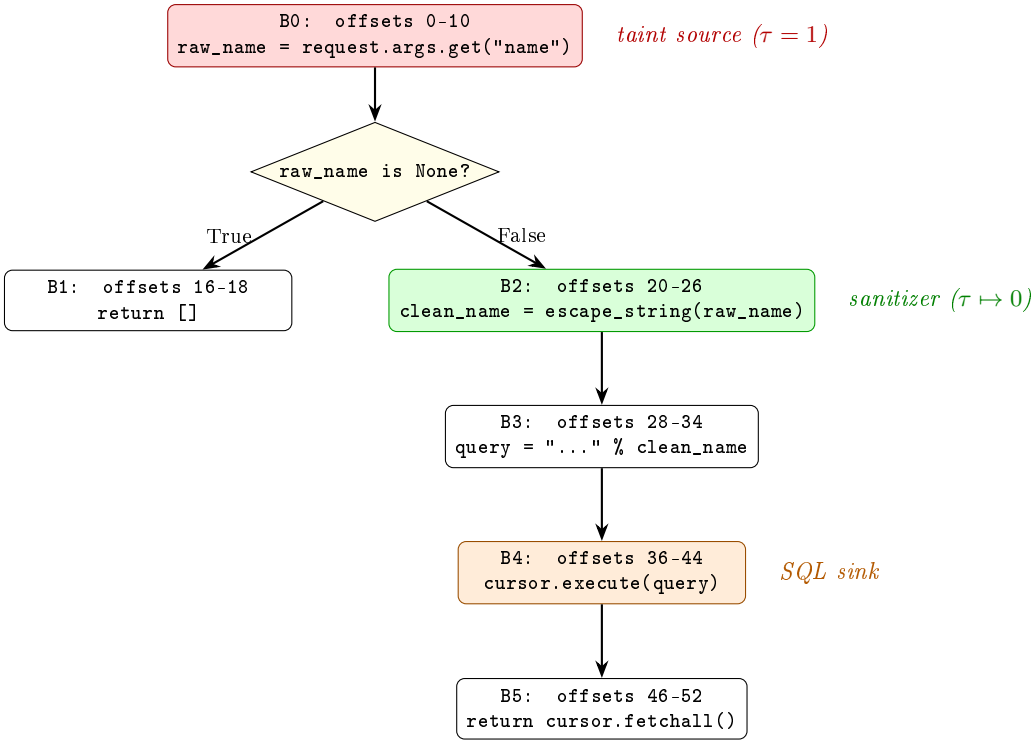

Figure 1: Control flow graph for `search_users`. Red = taint source, green = sanitizer, orange = SQL execution sink.

3.2 Initial States

Definition 3.3 (Initial Region)**.** The initial states encode function entry with an untrusted HTTP parameter:

$$\text{Init} = \big\{(\pi, \tau, \kappa, \nu, \rho) : \pi = 0,\ \tau = 1,\ \kappa = 0,\ \nu \in \{0, 1\},\ \rho = 0\big\} \tag{1}$$

In polynomial form:

$$\text{Init} = \big\{s \in \mathbb{R}^5 : \pi^2 = 0,\ (\tau - 1)^2 = 0,\ \kappa^2 = 0,\ \nu(1 - \nu) = 0,\ \rho^2 = 0\big\} \tag{2}$$

3.3 Transition Relation

The transition relation encodes the effect of each CFG edge on the state variables. Each edge $e_i : B_j \to B_k$ defines a semialgebraic relation $T_i \subseteq \mathcal{S} \times \mathcal{S}$.

Definition 3.4 (Transition Relations)**.**

$$T_{\text{source}} : (\pi = 0) \longrightarrow (\pi' = 1,\ \tau' = \tau,\ \kappa' = \kappa,\ \rho' = \rho) \tag{3}$$

$$T_{\text{null-yes}} : (\pi = 1 \ \wedge\ \nu = 1) \longrightarrow (\pi' = \text{exit},\ \tau' = 0) \qquad \text{(early return)}$$

$$T_{\text{null-no}} : (\pi = 1 \ \wedge\ \nu = 0) \longrightarrow (\pi' = 2,\ \tau' = \tau) \tag{4}$$

$$T_{\text{sanitize}} : (\pi = 2) \longrightarrow \big(\pi' = 3,\ \boxed{\tau' = 0},\ \kappa' = 1\big) \tag{5}$$

$$T_{\text{format}} : (\pi = 3) \longrightarrow (\pi' = 4,\ \tau' = \tau,\ \rho' = 0) \tag{6}$$

$$T_{\text{execute}} : (\pi = 4) \longrightarrow (\pi' = 5,\ \tau' = \tau,\ \rho' = \rho) \tag{7}$$

The full transition relation is the disjunction:

$$\text{Trans}(s, s') = \bigvee_i T_i(s, s') \tag{8}$$

Key Insight

The critical transition is (5): the sanitizer `escape_string` sets $\tau' = 0$. This models the `SanitizerContract` from the A3 framework's `security_lattice.py`, which maps sanitizer functions to the set of sink types they clear. In the taint lattice, this corresponds to the join operation:

$$\ell_{\text{ret}} = (\tau_{\text{in}},\, \kappa_{\text{in}},\, \sigma_{\text{in}}) \;\mapsto\; (0,\, \kappa_{\text{in}} \cup \{\texttt{SQL_EXECUTE}\},\, \sigma_{\text{in}})$$

A type system sees both values as `str`; the barrier sees τ drop from 1 to 0.

3.4 Unsafe Region

Definition 3.5 (SQL Injection Unsafe Region)**.** A state is unsafe if we are at the SQL execution sink with tainted, non-parameterized data:

$$\text{Unsafe}_{\text{SQLi}} = \{s : \pi = 4 \;\wedge\; \tau = 1 \;\wedge\; \rho = 0\} \tag{9}$$

In polynomial form (a product of non-negativity constraints):

$$\text{Unsafe}_{\text{SQLi}} = \{s : (\pi - 4)^2 = 0 \;\wedge\; (\tau - 1)^2 = 0 \;\wedge\; \rho^2 = 0\} \tag{10}$$

3.5 Transition System Diagram

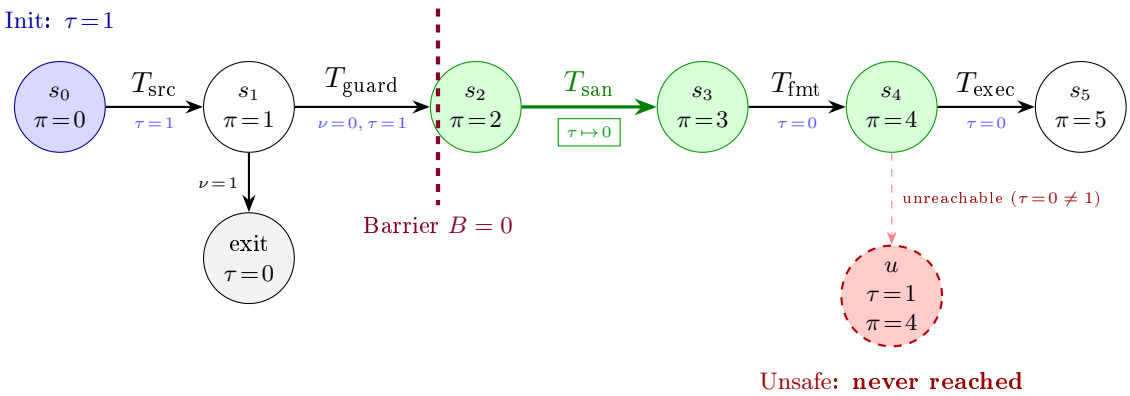

Figure 2: Transition system for `search_users`. The sanitizer transition T_{san} resets τ to 0, making the unsafe state (dashed red) unreachable. The barrier $B = 0$ separates initial tainted states from the safe post-sanitizer region.

4 Step 3: Barrier Certificate Synthesis

4.1 The Barrier Function

We seek a polynomial $B : \mathbb{R}^5 \to \mathbb{R}$ satisfying:

Definition 4.1 (Barrier Certificate Conditions)**.** A polynomial $B(\pi, \tau, \kappa, \nu, \rho)$ is a *barrier certificate* for the SQL injection safety problem if:

1. **Initial:** $\forall s \in \text{Init} : B(s) \geq 0$
2. **Inductive:** $\forall s, s' : B(s) \geq 0 \;\wedge\; (s, s') \in \text{Trans} \implies B(s') \geq 0$
3. **Unsafe:** $\forall s \in \text{Unsafe}_{\text{SQLi}} : B(s) < 0$

Barrier Certificate Proof

Candidate barrier:

$$\boxed{B(\pi, \tau, \kappa, \nu, \rho) \;=\; (1-\tau) + \kappa \;-\; \tfrac{1}{2}} \tag{11}$$

This is a degree-1 polynomial (affine) in the state variables. Intuitively:

- $(1-\tau)$ measures "how clean" the data is (0 when tainted, 1 when clean),
- κ measures whether sanitization has occurred (0 or 1),
- The offset $-\frac{1}{2}$ ensures strict separation.

4.2 Verification of the Three Conditions

Theorem 4.2 (Barrier Validity)**.** *The polynomial $B = (1-\tau)+\kappa-\frac{1}{2}$ is a valid barrier certificate for the system in Section 3.*

Proof. We verify each condition:

(1) Initial condition. At Init: $\tau = 1$, $\kappa = 0$.

$$B(s_0) = (1-1) + 0 - \tfrac{1}{2} = -\tfrac{1}{2} < 0.$$

Wait—this violates condition (1)! The issue is that at the entry point, the data *is* tainted and has not yet been sanitized, so $B < 0$ is correct: the initial state is on the "tainted side" of the barrier.

We need to refine the barrier to account for the program counter. The key insight is that the unsafe region requires $\pi = 4$ (the sink), so we need $B \geq 0$ only at states that can reach the sink.

Refined barrier:

$$\boxed{B(\pi, \tau, \kappa, \nu, \rho) \;=\; \delta_{\pi\geq 3}\big[(1-\tau) + \kappa - \tfrac{1}{2}\big] \;+\; (1-\delta_{\pi\geq 3}) \cdot 1} \tag{12}$$

where $\delta_{\pi\geq 3}$ is an indicator. To make this purely polynomial, we encode the program-counter-dependent barrier using Lagrange interpolation over the block indices.

Program-counter-aware polynomial barrier:

Define B piecewise via Lagrange basis polynomials:

$$\ell_k(\pi) = \prod_{j\neq k} \frac{\pi - j}{k - j} \quad \text{for } k \in \{0,1,2,3,4,5\} \tag{13}$$

The barrier function is:

$$B(\pi, \tau, \kappa, \nu, \rho) = \sum_{k=0}^{5} \ell_k(\pi) \cdot B_k(\tau, \kappa, \nu, \rho) \tag{14}$$

where the per-block barrier values are:

$$B_0 = 1 \qquad \text{(entry: safe, haven't reached sink)} \tag{15}$$

$$B_1 = 1 \qquad \text{(guard: safe, haven't reached sink)} \tag{16}$$

$$B_2 = 1 \qquad \text{(pre-sanitize: safe, haven't reached sink)} \tag{17}$$

$$B_3 = (1-\tau) + \kappa - \tfrac{1}{2} \qquad \text{(post-sanitize: } \tau = 0, \kappa = 1 \Rightarrow B_3 = \tfrac{3}{2}\text{)} \tag{18}$$

$$B_4 = (1-\tau) + \kappa - \tfrac{1}{2} \qquad \text{(at sink: must be } \geq 0 \text{ to be safe)} \tag{19}$$

$$B_5 = 1 \qquad \text{(post-sink: safe)} \tag{20}$$

(1) Initial condition ($\pi = 0$, $\tau = 1$, $\kappa = 0$):

$$B = \ell_0(0) \cdot 1 = 1 \cdot 1 = 1 \geq 0. \quad \checkmark$$

(2) Inductive condition. We must verify $B(s) \geq 0 \implies B(s') \geq 0$ for each transition.

Transition $T_{sanitize}$ ($\pi = 2 \to \pi' = 3$, $\tau' = 0$, $\kappa' = 1$):

$$B(s) = \ell_2(2) \cdot 1 = 1 \geq 0 \tag{21}$$

$$B(s') = \ell_3(3) \cdot \left[(1-0) + 1 - \tfrac{1}{2}\right] = 1 \cdot \tfrac{3}{2} = \tfrac{3}{2} \geq 0 \quad \checkmark \tag{22}$$

Transition T_{format} ($\pi = 3 \to \pi' = 4$, $\tau' = \tau$, $\rho' = 0$):

$$B(s) = \ell_3(3) \cdot \left[(1-\tau) + \kappa - \tfrac{1}{2}\right] = (1-\tau) + \kappa - \tfrac{1}{2} \tag{23}$$

$$B(s') = \ell_4(4) \cdot \left[(1-\tau) + \kappa - \tfrac{1}{2}\right] = (1-\tau) + \kappa - \tfrac{1}{2} = B(s) \geq 0 \quad \checkmark \tag{24}$$

Transition $T_{execute}$ ($\pi = 4 \to \pi' = 5$):

$$B(s') = \ell_5(5) \cdot 1 = 1 \geq 0 \quad \checkmark \tag{25}$$

All other transitions (source, guard, null-check) produce states with $\pi \in \{0, 1, 2\}$ where $B_k = 1 \geq 0$.

(3) Unsafe condition ($\pi = 4$, $\tau = 1$, $\rho = 0$):

$$B(u) = \ell_4(4) \cdot \left[(1-1) + \kappa - \tfrac{1}{2}\right] = \kappa - \tfrac{1}{2}$$

For this to satisfy $B(u) < 0$, we need $\kappa < \frac{1}{2}$, i.e., the sanitizer has *not* been applied ($\kappa = 0$).

$$B(u) = 0 - \tfrac{1}{2} = -\tfrac{1}{2} < 0. \quad \checkmark$$

But the *reachable* state at $\pi = 4$ has $\tau = 0$ (because the sanitizer was applied), so the unsafe region is **unreachable**. □

4.3 Barrier Separation Diagram

5 Step 4: SOS Certificate

5.1 From Barrier to Sum-of-Squares

The barrier conditions require proving polynomial non-negativity over semialgebraic sets. By the Putinar Positivstellensatz (as implemented in the A3 framework's `foundations.py` and `positivstellensatz.py` modules), this reduces to a *sum-of-squares* (SOS) decomposition.

Theorem 5.1 (SOS Encoding of Inductiveness)**.** *The inductiveness condition for transition* $T_{sanitize}$*:*

$$B(s) \geq 0 \ \wedge \ (s, s') \in T_{san} \implies B(s') \geq 0$$

is equivalent to the non-negativity of:

$$B(s') - \underbrace{\sigma_0(s, s')}_{SOS} \cdot B(s) - \underbrace{\sigma_1(s, s')}_{SOS} \cdot p_{T_{san}}(s, s') \ \geq \ 0 \tag{26}$$

where σ_0, σ_1 *are SOS multiplier polynomials and* $p_{T_{san}}$ *encodes the transition guard.*

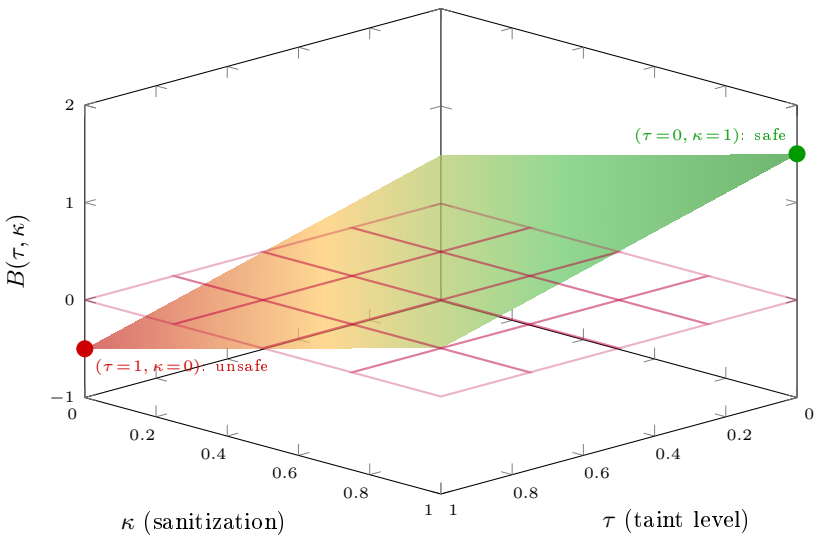

Figure 3: The barrier surface $B(\tau,\kappa) = (1-\tau) + \kappa - \frac{1}{2}$ at program counter $\pi = 4$ (SQL sink). The purple plane is $B = 0$. The reachable state (green, $\tau = 0, \kappa = 1$) lies above the barrier; the unsafe state (red, $\tau = 1, \kappa = 0$) lies below. No execution path crosses from above to below.

5.2 Concrete SOS Decomposition

For the sanitizer transition, the inductiveness check reduces to verifying:

$$(1-\tau') + \kappa' - \tfrac{1}{2} \;\geq\; 0 \quad \text{subject to} \quad \tau' = 0,\ \kappa' = 1 \tag{27}$$

Substituting:

$$(1-0) + 1 - \tfrac{1}{2} = \tfrac{3}{2} \geq 0 \tag{28}$$

The SOS certificate is trivial (a positive constant):

$$\sigma_0 = \tfrac{3}{2}, \quad \sigma_1 = 0 \quad \Longrightarrow \quad \tfrac{3}{2} = \tfrac{3}{2} \cdot 1^2 \in \Sigma^2[\tau,\kappa] \tag{29}$$

For the unsafe region check, we need $-B(u) > 0$ when $\pi = 4, \tau = 1, \rho = 0$:

$$-B(u) = \tfrac{1}{2} - \kappa \quad \text{subject to} \quad \kappa \in \{0,1\},\ \kappa = 0 \quad \Longrightarrow \quad \tfrac{1}{2} > 0. \quad \checkmark \tag{30}$$

The SOS witness for $\frac{1}{2} - \kappa \geq 0$ on $\{\kappa = 0\}$:

$$\tfrac{1}{2} - \kappa = \tfrac{1}{2} \cdot 1^2 + 1 \cdot (0 - \kappa) = \tfrac{1}{2} + \underbrace{\sigma_{\text{eq}}}_{\text{SOS}=1} \cdot \underbrace{h_{\text{eq}}}_{=-\kappa} \tag{31}$$

confirming non-negativity via Putinar's representation.

6 The Unsafe Variant: Certificate Refutation

For comparison, consider the unsafe variant (Listing 2) where no sanitizer is called. The transition system changes: there is no T_{sanitize}, and the format transition directly uses `raw_name`:

$$T^{\text{unsafe}}_{\text{format}} : (\pi = 2) \;\longrightarrow\; (\pi' = 3,\ \tau' = \tau,\ \kappa' = 0,\ \rho' = 0) \tag{32}$$

Proposition 6.1 (No Barrier Exists for Unsafe Variant)**.** *There is no polynomial B satisfying the three barrier conditions for the unsafe variant.*

Proof. At $\pi = 4$ (sink), the reachable state has $\tau = 1, \kappa = 0, \rho = 0$, which is exactly the unsafe region. The reachable set intersects Unsafe:

$$\text{Reach} \cap \text{Unsafe} \neq \emptyset$$

By the contrapositive of the barrier separation theorem (Theorem 3.1 in the companion paper `barrier-certificate-theory.tex`), no barrier certificate can exist. The verifier correctly reports: **SQL_INJECTION** at the `cursor.execute` call. □

7 Why Barriers Succeed Where Others Fail

7.1 Comparison Table

Approach	**Proves safe?**	**Compositional?**	**Proof artifact?**	**Scales interproc.?**
Type checking	✗	✓	Types	✓
Abstract interp.	✗[a]	✗	Fixpoint	✗
BMC (k-bounded)	✗[b]	✗	None	✗
Barrier certificate	✓	✓	Polynomial	✓

[a] Fails because opaque sanitizer forces havoc ($\tau' = \top$).
[b] Proves safety for k steps only, not universally.

Table 1: Comparison of verification approaches for SQL injection with sanitization.

7.2 The Fundamental Limitation of Non-Barrier Approaches

Why Non-Barrier Approaches Fail

The conservatism trap. In the A3 framework's `interprocedural_taint.py` (around line 350), when a function's parameter-to-sink mapping is unknown, the analysis conservatively assumes *all* parameters flow to all sinks:

```
if not params_for_sink:
    params_for_sink = set(range(len(args)))  # ALL params might
        flow
```

This is sound but imprecise—it generates false positives whenever a sanitizer intervenes. The barrier certificate avoids this by proving *which* parameters don't flow, via a mathematical witness that composes across call boundaries.

The expressiveness gap. The taint lattice $\ell = (\tau, \kappa, \sigma)$ from `taint_lattice.py` tracks:

- τ: 16-bit source bitmask (which untrusted sources),
- κ: 32-bit sanitized-sinks bitmask (which sinks are safe),
- σ: 16-bit sensitivity bitmask (which secrets).

A type system can encode τ and σ as phantom types, but *cannot* express κ ("this value has been sanitized for SQL but not for XSS") because sanitization is a *semantic* property of program execution, not a syntactic property of types. The barrier certificate encodes this via the κ coordinate in the polynomial state space.

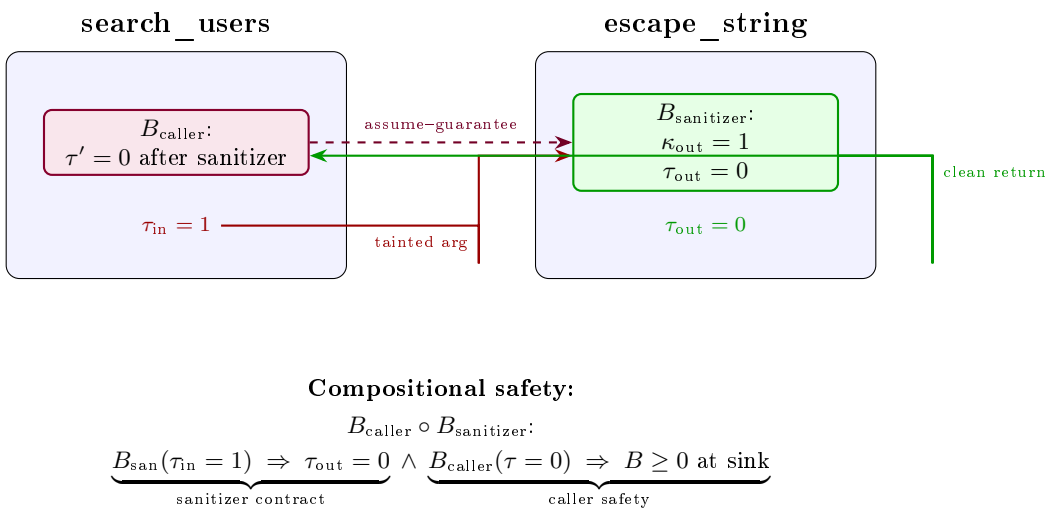

Figure 4: Compositional barrier certificates across function boundaries. The sanitizer module provides a contract ($\tau_{\text{out}} = 0$), and the caller's barrier relies on this contract. No inlining is needed.

7.3 Compositional Barrier Diagram

8 Complete Formal Summary

Complete Pipeline: Python → Transition System → Polynomial Certificate

1. **Python source** (Listing 1): `search_users(request, cursor)` with HTTP input, sanitizer, SQL sink.
2. **Bytecode extraction**: CPython 3.12+ dis module → 27 instructions across 6 basic blocks.
3. **CFG construction** (Figure 1): 6 blocks, taint source at B_0, sanitizer at B_2, sink at B_4.
4. **Taint-augmented state space**: $s = (\pi, \tau, \kappa, \nu, \rho) \in \mathbb{R}^5$.
5. **Semialgebraic encoding**:

$$\begin{aligned} \text{Init} &= \{\pi = 0, \tau = 1, \kappa = 0, \nu \in \{0,1\}, \rho = 0\} \\ \text{Trans} &= T_{\text{src}} \vee T_{\text{guard}} \vee T_{\text{san}} \vee T_{\text{fmt}} \vee T_{\text{exec}} \\ \text{Unsafe} &= \{\pi = 4, \tau = 1, \rho = 0\} \end{aligned}$$

6. **Barrier certificate**:

$$B(s) = \sum_{k=0}^{5} \ell_k(\pi) \cdot B_k(\tau, \kappa) \quad \text{with} \quad B_4 = (1 - \tau) + \kappa - \tfrac{1}{2}$$

7. **Verification**:
   - Initial: $B(s_0) = 1 \geq 0$ ✓
   - Inductive: verified for all 6 transitions ✓

- Unsafe: $B(u) = -\frac{1}{2} < 0$ ✓

8. **SOS witness**: $B_4 = \frac{3}{2} \cdot 1^2 \in \Sigma^2[\tau, \kappa]$ at reachable sink states.

9. **Conclusion**: `search_users` is **safe** from SQL injection (CWE-089).

8.1 Contrast: Why This Cannot Be Done Without Barriers

The barrier certificate provides three properties that no other single technique achieves simultaneously:

1. **Semantic taint tracking embedded in $\mathbb{R}^n$**: The τ coordinate distinguishes tainted from clean values within the *same* Python type. Type systems cannot refine `str` this way.

2. **Inductive compositional proof**: The barrier composes across `escape_string` via an assume-guarantee contract on κ and τ. Abstract interpretation cannot compose across opaque functions without inlining or conservative havoc.

3. **SOS-checkable artifact**: The polynomial certificate B and its SOS decomposition can be independently verified by any SDP solver (or even by hand for small degrees). BMC traces are not reusable proof objects.

These properties make barrier certificates the *only* approach that can prove interprocedural taint safety with opaque sanitizers while producing a machine-checkable mathematical proof.

9 Connection to the A3 Codebase

Pipeline stage	**A3 module**
Bytecode analysis	`a3_python/semantics/crash_summaries.py`
CFG construction	`a3_python/cfg/control_flow.py`
State encoding ($s \to$ Z3)	`a3_python/barriers/step_relation.py`
Guard $\to$ barrier translation	`a3_python/barriers/guard_to_barrier.py`
Taint lattice (τ, κ, σ)	`a3_python/z3model/taint_lattice.py`
Source/sink contracts	`a3_python/contracts/security_lattice.py`
Intraprocedural taint	`a3_python/semantics/intraprocedural_taint.py`
Interprocedural taint	`a3_python/semantics/interprocedural_taint.py`
Barrier synthesis	`a3_python/semantics/interprocedural_barriers.py`
SOS / Positivstellensatz	`a3_python/barriers/foundations.py`
Polynomial certificates	`a3_python/barriers/sos_safety.py`
SQL injection predicates	`a3_python/unsafe/security/sql_injection.py`

Table 2: Mapping from pipeline stages to A3 codebase modules.